M10 # Skills

Methodology Intermediate

Reusable skill modules with embedded resources

Skill structure

A skill is a folder in `.claude/skills/` containing a `SKILL.md` file and optionally additional resources (docs, templates, scripts, checklists).

```
my-skill/
├── SKILL.md          # Instructions + frontmatter (required)
├── reference.md      # Reference documentation
├── checklists/
│   └── quality.md
└── examples/
    └── pattern.ts
```

Invocation modes

Mode	Frontmatter	Behavior
User-invocable	`disable-model-invocation: true`	Triggered by user via `/name`
Model-invocable only	`user-invocable: false`	Hidden from the user menu; Claude may invoke it
Both	(no flag)	Invocable either way

SKILL.md frontmatter

```
---
name: security-guardian
description: OWASP expertise, auth, and data protection
allowed-tools: Read Grep Bash
---
```

Field	Role
`name`	Kebab-case, 1-64 chars, must match the folder
`description`	Activation trigger; with `when_to_use`, truncated at 1,536 chars in the listing (1,024 is the agentskills.io limit)
`allowed-tools`	Tool whitelist (supports wildcards)
`disable-model-invocation`	`true` for manual-only invocation
`user-invocable`	`false` hides the skill from the `/` menu
`model` / `effort`	Per-skill model (`inherit` allowed) and effort (`low` to `max`)
`context` / `paths`	`fork` runs in a subagent; `paths` globs limit activation

Skills vs commands

Aspect	Skill	Command
Resources	Can embed files	Markdown only
Scope	Domain knowledge	Procedural workflow
Reuse	Between agents	Direct invocation
Standard	agentskills.io (portable)	Claude Code only

The main difference: a skill can embed a 500-line `reference.md` file that becomes available on invocation. A command cannot.

Installing community skills

```
# Inspect available skills before installing
npx skills add vercel-labs/agent-skills --list

# Select the desired skills interactively
npx skills add vercel-labs/agent-skills --agent claude-code

# Check installed skills
npx skills list
```

The skills CLI installs community packages. Listing or scanning does not guarantee safety: inspect instructions, scripts, requested access and the chosen revision before use.

Invocation

Skills are triggered like commands, by their name:

```
/security-guardian        # Direct invocation
/tdd                      # TDD skill
/design-patterns detect   # With argument
```

Agents can preload skills via the `skills:` field in their frontmatter. The content is injected into the agent's context at startup.

Lifecycle and improvement

Two long-term management patterns:

Claudeception: meta-skill that generates new skills from session discoveries. Install:

`git clone https://github.com/blader/Claudeception ~/.claude/skills/claudeception`

Claude Reflect System: improves existing skills by analyzing corrections detected during sessions. Proposes changes before applying them.

Important caveat

Claude may invoke eligible skills automatically, but discovery does not guarantee invocation on every relevant request. Put critical always-applicable instructions in `CLAUDE.md` or `.claude/rules/` and test skill activation on representative prompts.